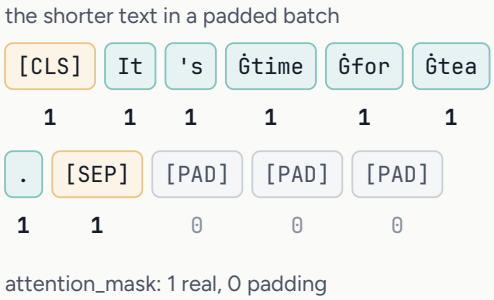
SECTION 1, PART 3 CHEAT SHEET

An encoder reads both ways, the kind that decides

An encoder sees every word at once, so each token's vector carries its whole sentence. Pretrained by filling in hidden words, then averaged and given a classification head, it decides.



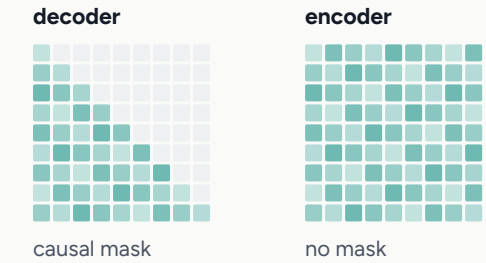
1 ENCODER TOKENS

[CLS], [SEP] and padding

The tokenizer adds `[CLS]` at the start and `[SEP]` at the end. A batch needs equal lengths: the shorter text gets `[PAD]`, which the attention mask keeps out.

```
MODEL = "answerdotai/ModernBERT-base"
tok = AutoTokenizer.from_pretrained(MODEL)
model = AutoModel.from_pretrained(MODEL,
    device_map="cuda")
x = tok(["It's time for tea.",
         "It's time for a cup of tea."], padding=True)
x["attention_mask"]
```

COURSE RUN **50,368** tokens of **768** numbers, **22** layers.



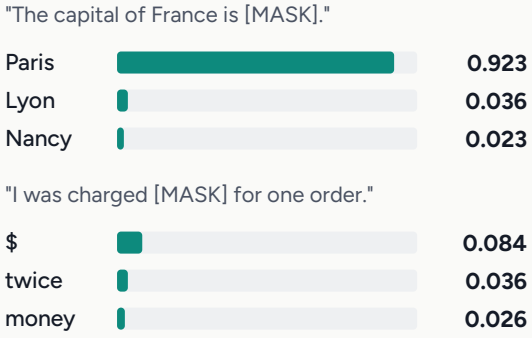
2 BOTH WAYS

No causal mask: a token sees what follows

A decoder looks only back; an encoder's attention covers the whole text, so a word can look at the words after it.

```
model.set_attn_implementation("eager")
response = model(**x, output_attentions=True)
len(response.attentions), response.attentions[0].shape
```

COURSE RUN **22** maps of **[1, 12, 13, 13]**: 12 heads over 13 tokens.



3 FILL IN [MASK]

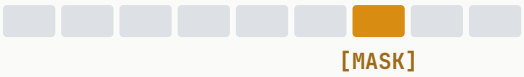
An encoder fills the gap

`fill-mask` scores every token for the hidden position, using the words on both sides.

```
fill = pipeline("fill-mask",
    model=MODEL, device_map="cuda")
fill("The capital of France is [MASK].")
```

COURSE RUN `Paris` **0.923**; for the charge, `$` **0.084** beat `twice`.

the labels, one per token



`-100` everywhere else: no loss there
`7785` at the gap: the id of `ĠParis`

loss 0.0798 -log p of Paris

4 HOW IT IS PRETRAINED

Masked language modelling: loss at the gap only

Hide a token; the loss is −log p of the real token at the gap. `-100` in the labels means no loss at that position.

```
gap = x["input_ids"] == tok.mask_token_id
paris = tok.convert_tokens_to_ids("ĠParis")
labels = torch.full_like(x["input_ids"], -100)
labels[gap] = paris
response = fill.model(**x, labels=labels)
response.loss
```

COURSE RUN loss **0.0798**.

the vector for Ġbank, sentence against sentence

	river	loan	fee
river	1.0000	0.7251	0.7256
loan	0.7251	1.0000	**0.9699**
fee	0.7256	**0.9699**	1.0000

the two money banks are the closest pair

5 CONTEXTUAL EMBEDDINGS

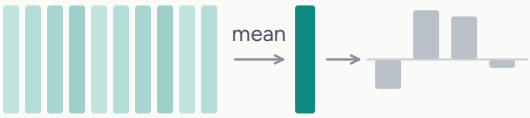
One word, a vector per sentence

`Ġbank` is token 2 in all three sentences; only the words after it tell them apart. A decoder at `bank` has seen only "The bank".

```
x = tok(texts, return_tensors="pt", padding=True).to("cuda")
response = model(**x)
bank = response.last_hidden_state[:, 2]
torch.cosine_similarity(bank[:, None], bank[None], dim=-1)
```

COURSE RUN loan against fee **0.9699**, river against loan **0.7251**.

one vector per token



`[1, 10, 768]` `[1, 768]` **4 scores**

Linear(768, 4) is new: its scores mean nothing until fine-tuning trains it with the encoder

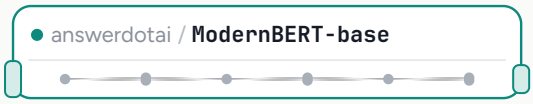
6 FROM TEXT TO DECISION

One vector, then a classification head

The mean over the tokens gives a sentence embedding. `AutoModelForSequenceClassification` adds a new layer with one score per label.

```
response.last_hidden_state.mean(dim=1).shape
clf = AutoModelForSequenceClassification.from_pretrained(
    MODEL, num_labels=4, device_map="cuda")
clf.head, clf.classifier
clf(**x).logits
```

COURSE RUN `clf.classifier` is **Linear(768, 4)**; `clf.head` is reused from pretraining.



AutoModel
no head: a vector per token **[1, 10, 768]**

fill-mask
word head: a score per word **Paris 0.923**

...ForSequenceClassification
new head: one score per label **4 scores**

7 PICK THE HEAD

One body, three heads

The pretrained encoder serves three jobs, and the class you load decides what comes out: a vector per token, word scores for a `[MASK]`, or one score per label.

```
model = AutoModel.from_pretrained(MODEL, device_map="cuda")
fill = pipeline("fill-mask", model=MODEL, device_map="cuda")
clf = AutoModelForSequenceClassification.from_pretrained(
    MODEL, num_labels=4, device_map="cuda")
```

COURSE RUN `last_hidden_state` **[1, 10, 768]**; the word head put Paris at **0.923**; the new head's top score, **0.4098**, means nothing until fine-tuning.

DECODER OR ENCODER: PARTS 2 AND 3 SIDE BY SIDE

	DECODER	ENCODER
Model in the course	`Qwen2.5-0.5B-Instruct`	`ModernBERT-base`
Looks at	earlier tokens only, the causal mask	the whole text, both ways
Pretrained by	predicting the next token	filling in `[MASK]`
Layers, heads	**24** layers, **14** heads	**22** layers, **12** heads
Vocabulary rows	**151,936**	**50,368**
Made for	writing	deciding

TERMS

encoder: reads the whole text both ways.

decoder: writes one token at a time.

pretraining: the training somebody else already did on the base model.

sentence embedding: one vector for a whole piece of text.

classification head: the Hugging Face layer that scores your labels.

WHAT EACH CALL HANDS BACK

`tok(texts, padding=True)`
input_ids and **attention_mask**

`fill(text)`
the top five: **score**, **token_str**, **sequence**

`model(**x).last_hidden_state`
text, tokens, numbers

`.mean(dim=1)`
one vector per text

`clf(**x).logits`
one score per label

WATCH OUT

- A new classification head's scores mean nothing yet: fine-tuning trains it and the encoder together.
- Pass the `attention_mask` with the ids, so attention never looks at `[PAD]`.